

AI Engineer Assignment: Build an Enterprise Knowledge Assistant

Overview

Your task is to design and build an **Enterprise Knowledge Assistant** that can answer questions from a collection of internal documents.

The goal is to build a practical AI application that demonstrates your ability to design, implement, and evaluate a production-oriented Retrieval Augmented Generation (RAG) system.

The solution should allow employees to ask questions in natural language and receive accurate answers based on the provided knowledge base.

Business Scenario

A company has hundreds of internal documents including:

- Product documentation
- HR policies
- Technical guides
- Customer FAQs
- Process documents
- Compliance guidelines

Employees currently spend significant time searching these documents manually.

Your task is to build an AI assistant that can:

- Understand employee questions
 - Retrieve relevant information from documents
 - Generate accurate answers
 - Provide references to the source information
-

Objective

Build a working AI-powered knowledge assistant with the following capabilities:

1. Document ingestion
 2. Knowledge extraction and indexing
 3. Semantic search
 4. Context-aware answer generation
 5. Source citation
 6. User interaction interface
-

Functional Requirements

1. Document Processing

The system should support ingestion of provided documents.

The pipeline should handle:

- Document loading
- Text extraction
- Chunking strategy
- Metadata management
- Embedding generation
- Storage in a searchable index

You should explain your design choices.

2. Question Answering

Users should be able to ask questions like:

Example:

Question:

What is the employee leave policy?

Expected Response:

Employees are eligible for 24 paid leaves annually.

Source:

- HR_Policy.pdf
- Page 12 source name and page number instead of context

The system should:

- Retrieve relevant information
 - Generate a concise answer
 - Avoid hallucinating information
 - Mention when information is unavailable
-

3. User Interface

Build a simple interface for interaction.

Acceptable options:

- Web application
- Chat interface Need to build an UI, streamlit or web application is preferred
- API endpoint
- Streamlit application

The interface should allow:

- Entering questions
 - Viewing responses
 - Viewing source documents/references
-

4. API (Optional but Preferred)

Expose your solution through an API.

Example:

POST `/ask`

Request:

```
{  
  "question": "What is the refund policy?"
```

```
}
```

Response:

```
{  
  "answer": "Refunds are allowed within 30 days.",  
  "sources": [  
    {  
      "document": "Customer_Policy.pdf",  
      "page": 5  
    }  
  ],  
  "confidence": 0.91  
}
```

Technical Expectations

You may use any technology stack.

Suggested tools:

Programming Language

- Python preferred

LLM

Examples:

- OpenAI models
- Anthropic models
- Llama models
- Other open-source models

Embeddings

Examples:

- OpenAI embeddings
- Sentence Transformers

- Other embedding models

Vector Database

Examples:

- FAISS
- ChromaDB
- Pinecone
- Weaviate
- Other alternatives

Frameworks

Optional:

- LangChain
- LlamaIndex
- Haystack

Evaluation Criteria

Your submission will be evaluated on:

1. AI System Design (25%)

We will evaluate:

- RAG architecture
- Retrieval strategy
- Chunking approach
- Embedding selection
- Prompt design

upload_pdf() [uploading pdf] ---> pdf stores in its directory ---> extracted the data in pdf by pdf_service ---> converting all the words in pdf into one string separated each in one line ---> next after extraction & conversion we convert it into chunks by chunk_text() in chunk_service ---> here the chunk size is 500 so it will see upto 500 letters/sentences ---> after completing of making all the pdfs into chunks we go into embeddings ---> in embeddings_service by using gemini embeddings convert all chunks into embeddings and store them ---> here faiss create indexes to all embeddings ---> after indexes map to chunks ---> when we give query it loads pdf indexes, chunks ---> next the query is converted into embeddings ---> then convert the query embeddings into array ---> then we will search the indexes from our query embeddings [k=5 it takes top 5] ---> then lastly we use llm that we copied api and using that prompt in it it will give the answer

2. Answer Quality (25%)

We will test:

- Accuracy

- Relevance
 - Handling of ambiguous questions
 - Hallucination prevention
-

3. Engineering Quality (20%)

We will evaluate:

- Code structure
 - Error handling
 - Maintainability
 - Scalability considerations
-

4. Evaluation Approach (15%)

Explain:

- How you measured performance
 - Test cases created
 - Improvements attempted
-

5. Documentation (15%)

Your README should include:

- Architecture overview
 - Setup instructions
 - Technology choices
 - Design decisions
 - Limitations
 - Future improvements
-

Bonus Features (Optional)

Additional points for implementing:

- Conversation memory
 - Hybrid search (keyword + semantic)
 - Query rewriting
 - Re-ranking
 - Multi-document reasoning
 - User feedback collection
 - Evaluation metrics
 - Authentication
 - Deployment
-

Submission Requirements

Please submit:

1. Source Code

Git repository containing:

- Application code
 - Configuration files
 - Requirements/dependencies
 - Sample data handling
-

2. README Documentation

Include:

- How to run the application
 - Architecture diagram
 - Technical decisions
 - Known limitations
-

3. Demo Video

5-minute maximum video showing:

- Application walkthrough

- Example queries
- Explanation of architecture

Video Submission Format:

Please upload your demo video to either:

- **Google Drive** (ensure link sharing permission is set to "Anyone with the link can view")
OR
 - **YouTube** (Unlisted videos are acceptable)
-

4. System Design Document

1–2 pages covering:

- High-level architecture
 - Data flow
 - Component explanation
 - Scalability considerations
-

Time Limit

You have **72 hours** from receiving this assignment to submit your solution.

The expected effort is approximately **12–20 hours**.

Important Notes

- Focus on quality over complexity.
- We value engineering judgment and clear explanations.
- Using existing libraries is allowed.
- Clearly mention any assumptions.
- Do not share confidential information or API keys in your submission.

Good luck!